
Finding Shortest Paths

Shortest Path Problem

Shortest Path Problem

We are given a graph $G = (V, E)$ and an edge weight function $\omega: E \rightarrow \mathbb{R}$.

Length of a Path

The *length* or *weight* $\omega(P)$ of a path $P = \{v_1, v_2, \dots, v_l\}$ with at least two vertices is

$$\omega(P) = \sum_{i=1}^{l-1} \omega(v_i v_{i+1})$$

If $|P| = 1$, $\omega(P) = 0$.

Shortest Path Problem

Shortest Path Problem

For two vertices u and v , the shortest path from u to v is the path P for which $\omega(P)$ is minimal. The *distance* $d(u, v)$ from u to v is the length of a shortest path from u to v .

Shortest Path Problem

Variants

- ▶ *Single Pair Shortest Path (SPSP)*
Find a shortest path from a vertex u to some vertex v .
- ▶ *Single Source Shortest Path (SSSP)*
Find shortest paths from a source vertex v to all other vertices in the graph.
- ▶ *All Pairs Shortest Path (APSP)*
Find shortest paths for all vertex pairs u and v .

There is no algorithm for SPSP which is asymptotically better in general than an algorithm for SSSP.

Shortest Path Properties

Theorem

Optimal Substructure Property

Each subpath of a shortest path is a shortest path.

Theorem

Triangle Inequality

For all vertices u , v , and w ,

$$d(u, v) \leq d(u, w) + d(w, v).$$

Negative Weight Edges and Cycles

Negative Weight Edges

- ▶ Natural in some application
- ▶ Makes finding a shortest path harder

Theorem

If there is a path from u to v containing a vertex w and w is in a cycle C with $\omega(C) < 0$, then there is no shortest path from u to v .

Avoiding Cycles

- ▶ Only permit simple paths, i. e., no vertex twice
- ▶ Follows if graph has no negative cycles
- ▶ With negative cycles, shortest simple path problem equal to longest simple path problem
- ▶ Problem: loss of optimal substructure property

General Approach

General Approach

Store for each vertex v

- ▶ $\text{dist}_s(v)$, length of currently best known path P from start vertex s to v
- ▶ $\text{par}_s(v)$, parent of v in P

Relaxation

- ▶ Updates best known distance.

```
1 Procedure RELAX( $u, v$ )  
2   If  $\text{dist}_s(v) > \text{dist}_s(u) + \omega(uv)$  Then  
3     Set  $\text{par}_s(v) := u$  and  $\text{dist}_s(v) := \text{dist}_s(u) + \omega(uv)$ .
```

General Approach

Initialization

- ▶ Set $\text{par}_s(v) := \text{null}$ and $\text{dist}_s(v) := \infty$ for each vertex v .
- ▶ Set $\text{dist}_s(s) := 0$ for start vertex s .

Iteration

- ▶ Pick vertex pair u, v .
- ▶ Call $\text{RELAX}(u, v)$
- ▶ Repeat

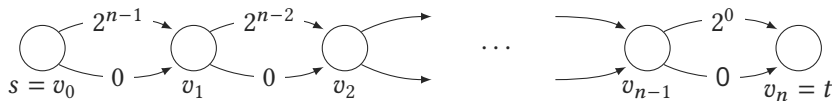
Open Questions

- ▶ How do we pick u and v ?
- ▶ When do we stop the iteration?

General Approach

Lemma

Calling $\text{RELAX}(\cdot, \cdot)$ in a “wrong” order may result in up to exponential runtime, even if each call improves some distance.



Approach

- ▶ When relaxing edges of u , relax the upper edge first.
- ▶ If relaxing uv updated v 's distance, recursively relax v 's edge (before finishing u 's).

Single Source Shortest Path

Observation

- ▶ A shortest path has at most $|V| - 1$ edges.
- ▶ If we know all shortest path with k edges, we can compute all shortest paths with $k + 1$ edges by relaxing all edges once.

```
1 For Each  $v \in V$ 
2   | Set  $\text{dist}(v) := \infty$  and  $\text{par}(v) = \text{null}$ .
3 Set  $\text{dist}(s) := 0$ .
4 For  $i := 1$  To  $|V| - 1$ 
5   | For Each  $(u, v) \in E$ 
6     | RELAX( $u, v$ )
```

Properties

- ▶ Runtime: $\mathcal{O}(|V||E|)$
- ▶ Works with negative weight edges
- ▶ Can detect negative cycles

Detecting negative cycles

- ▶ Negative cycle $\rightarrow$ There is always an edge (u, v) for which $\text{RELAX}(u, v)$ updates $\text{dist}(v)$.
- ▶ If $\text{RELAX}(u, v)$ still updates $\text{dist}(v)$ for $i \geq |V|$, then (u, v) is part of a negative cycle.

Dijkstra's Algorithm

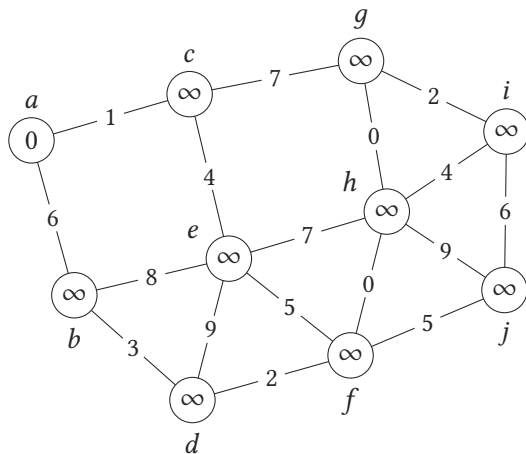
Idea

- ▶ Let S be set of vertices where shortest path is known.
- ▶ Relax all outgoing edges uv , i. e., $u \in S$ and $v \notin S$.
- ▶ Pick $v \notin S$ such that $\text{dist}(v)$ is minimal.
- ▶ For that v , $\text{dist}(v)$ is optimal. Add v to S and repeat.

Dijkstra's Algorithm

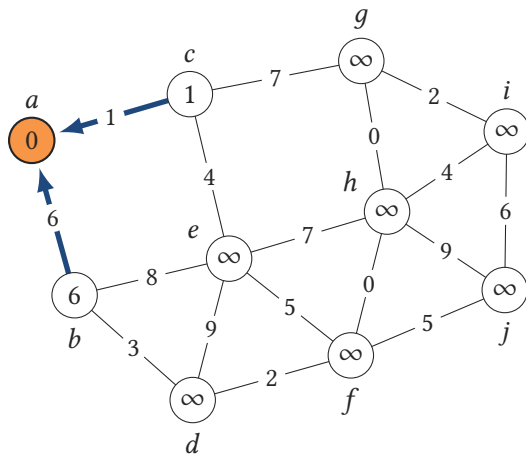
```
1 Initialize( $G, s$ )
2 Create a priority queue  $Q$  and add all vertices in  $V$ .
3 While  $Q$  is not empty
4   Remove  $v$  with minimal  $\text{dist}(v)$  from  $Q$ .
5   For Each  $(v, w) \in E$ 
6     RELAX( $v, w$ )
```

Dijkstra's Algorithm – Example



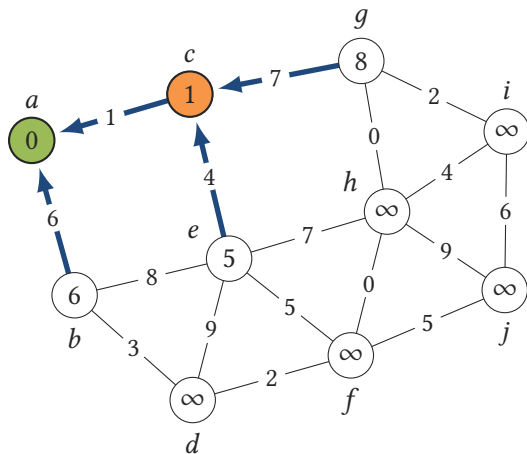
$Q:$ $a: 0$ $b: \infty$ $c: \infty$ $d: \infty$ $e: \infty$ $f: \infty$ $g: \infty$ $h: \infty$ $i: \infty$ $j: \infty$

Dijkstra's Algorithm – Example



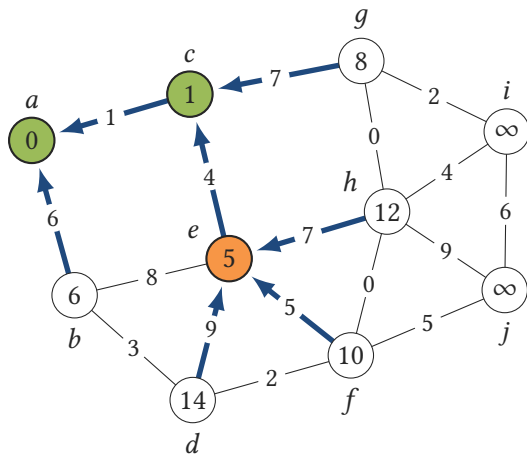
Q : $c: 1$ $b: 6$ $d: \infty$ $e: \infty$ $f: \infty$ $g: \infty$ $h: \infty$ $i: \infty$ $j: \infty$

Dijkstra's Algorithm – Example



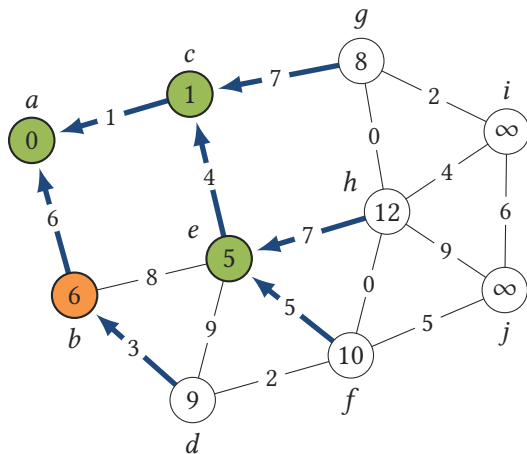
$Q: e: 5 \quad b: 6 \quad g: 8 \quad d: \infty \quad f: \infty \quad h: \infty \quad i: \infty \quad j: \infty$

Dijkstra's Algorithm – Example



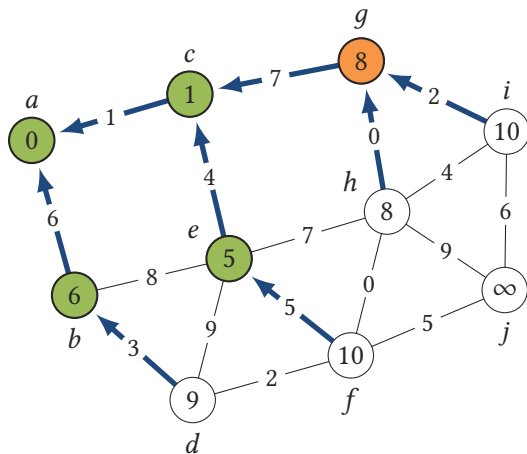
Q : $b: 6$ $g: 8$ $f: 10$ $h: 12$ $d: 14$ $i: \infty$ $j: \infty$

Dijkstra's Algorithm – Example



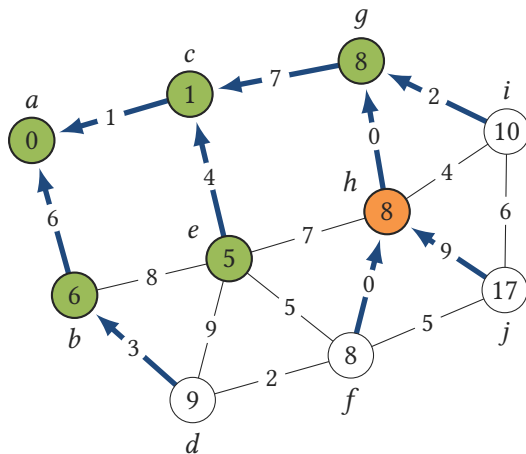
Q: $g: 8$ $d: 9$ $f: 10$ $h: 12$ $i: \infty$ $j: \infty$

Dijkstra's Algorithm – Example



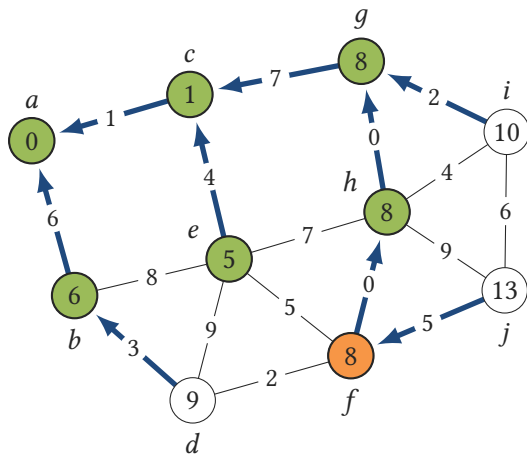
$Q: \quad h: 8 \quad d: 9 \quad f: 10 \quad i: 10 \quad j: \infty$

Dijkstra's Algorithm – Example



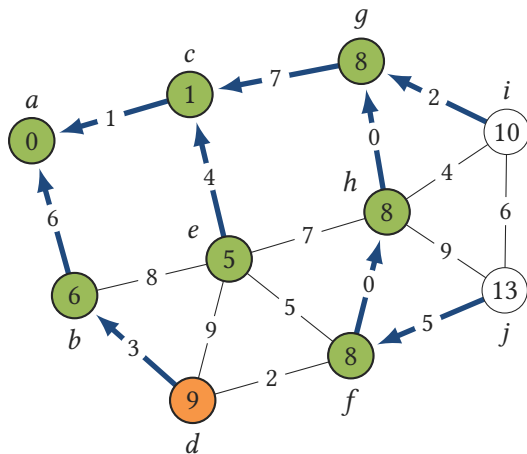
$Q: f: 8 \quad d: 9 \quad i: 10 \quad j: 17$

Dijkstra's Algorithm – Example



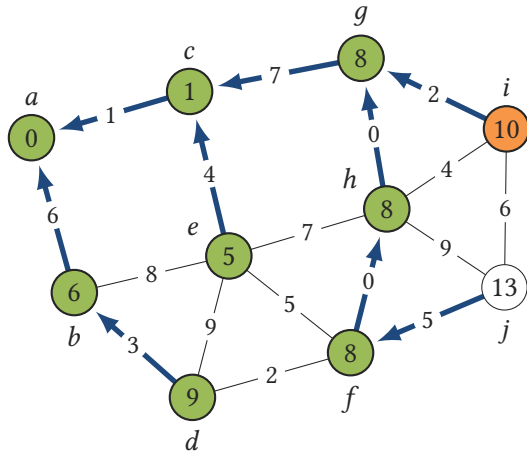
$Q: d: 9 \quad i: 10 \quad j: 13$

Dijkstra's Algorithm – Example



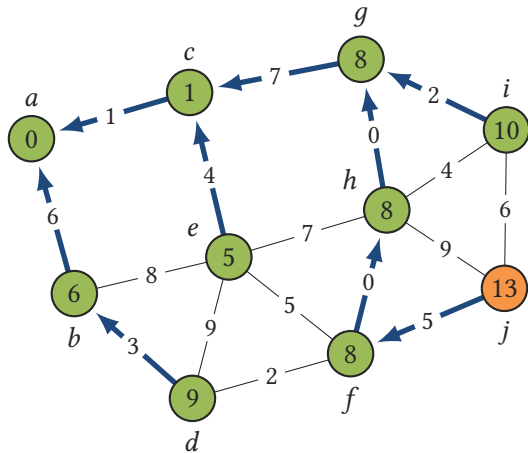
$Q: i: 10 \quad j: 13$

Dijkstra's Algorithm – Example



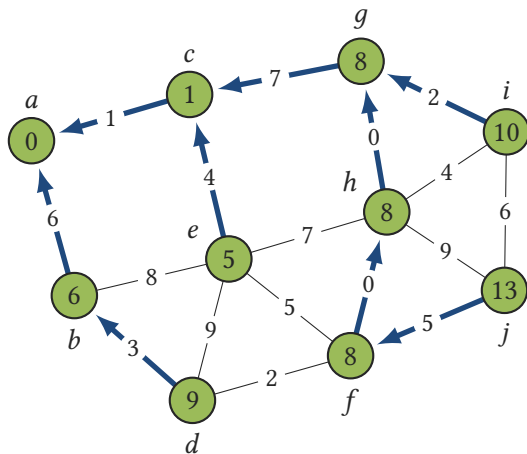
$Q: j: 13$

Dijkstra's Algorithm – Example



Q:

Dijkstra's Algorithm – Example



Q:

Dijkstra's Algorithm

Properties

- ▶ Runtime: $O(|E| \log |V|)$ with binary heaps and $O(|V| \log |V| + |E|)$ with Fibonacci-Heaps
- ▶ Invariant: For all vertices in S , $\text{dist}(s)$ is optimal.
- ▶ Requirement: No negative edges. The algorithm assumes that distances are always increasing.

What happens if there are negative edges?

